

# **Лабораторная работа №7**

**Имитационное моделирование**

Чистов Даниил Максимович

# Содержание

|          |                                  |           |
|----------|----------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>               | <b>5</b>  |
| <b>2</b> | <b>Выполнение работы</b>         | <b>6</b>  |
| 2.1      | Предварительные работы . . . . . | 6         |
| 2.2      | О модели M/M/c . . . . .         | 6         |
| 2.3      | О модели Росса . . . . .         | 12        |
| <b>3</b> | <b>Выводы</b>                    | <b>31</b> |
| <b>4</b> | <b>Список литературы</b>         | <b>32</b> |

## Список иллюстраций

|     |                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------|----|
| 2.1 | Рабочее пространство для лабораторной работы . . . . .                                             | 6  |
| 2.2 | График динамики числа клиентов на протяжении всего прогона модели M/M/c . . . . .                  | 12 |
| 2.3 | График динамики числа исправных машин на протяжении всего прогона модели Росса . . . . .           | 28 |
| 2.4 | Начало графика динамики числа исправных машин на протяжении всего прогона модели Росса . . . . .   | 29 |
| 2.5 | Середина графика динамики числа исправных машин на протяжении всего прогона модели Росса . . . . . | 29 |
| 2.6 | Статистика прогона модели Росса . . . . .                                                          | 30 |

## Список таблиц

```
import Pkg
Pkg.activate("../project")
Pkg.instantiate()
```

Activating project at `C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab07\project`

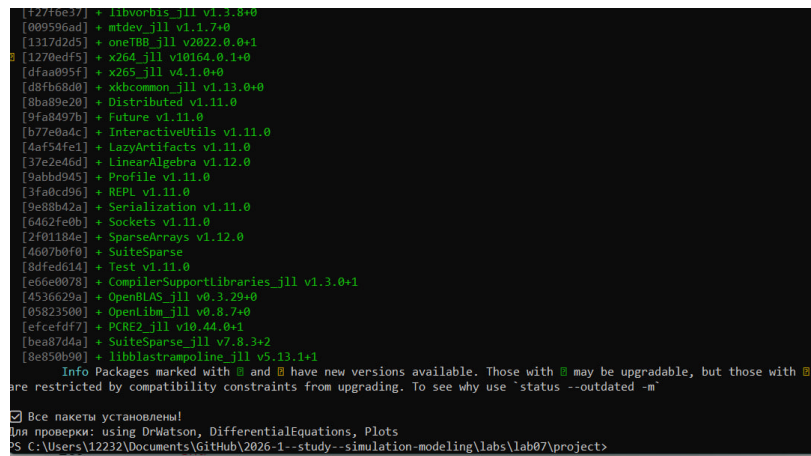
# 1 Цель работы

Целью данной работы является изучение моделей  $M/M/c$  и Росса, а также самостоятельная адаптация кодов моделей к структуре DrWatson, создание дополнительных графиков.

## 2 Выполнение работы

### 2.1 Предварительные работы

Перед началом работы с моделью создаю рабочее пространство для лабораторной работы (рис. 2.1).



```
[f2776e37] * libworris_jll v1.3.0+0
[009596ad] * mtdev_jll v1.1.7+0
[1317d2d5] * oneTBB_jll v2022.0.0+1
[1270edf5] * x264_jll v10164.0.1+0
[dfaa095f] * x265_jll v4.1.0+0
[d8fb68d0] * xkbcommon_jll v1.13.0+0
[8ba89e20] * Distributed v1.11.0
[9fa8497b] * Future v1.11.0
[b77e9a4c] * InteractiveUtils v1.11.0
[4af54fe1] * lazyArtifacts v1.11.0
[37e2e46d] * linearAlgebra v1.12.0
[9abb9945] * Profile v1.11.0
[3fa0cd96] * REPL v1.11.0
[9e88b42a] * Serialization v1.11.0
[6462fe0b] * Sockets v1.11.0
[2f01184e] * SparseArrays v1.12.0
[4607b0f0] * SuiteSparse
[8dfed614] * Test v1.11.0
[e66e0078] * CompilerSupportLibraries_jll v1.3.0+1
[4536629a] * OpenBLAS_jll v0.3.29+0
[05823500] * OpenLibm_jll v0.8.7+0
[efcefd77] * PCRE2_jll v10.44.0+1
[bca87d4a] * SuiteSparse_jll v7.8.3+2
[8e850b90] * libblastrampoline_jll v5.13.1+1

Info Packages marked with [!] and [!] have new versions available. Those with [!] may be upgradable, but those with [!]
are restricted by compatibility constraints from upgrading. To see why use `status --outdated -m`

[ ] Все пакеты установлены!
Для проверки: using DrWatson, DifferentialEquations, Plots
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab07\project>
```

Рисунок 2.1: Рабочее пространство для лабораторной работы

### 2.2 О модели М/М/с

Модель М/М/с (по классификации Кендалла) — это система массового обслуживания со следующими свойствами:

- М (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром  $\lambda$ .

- $M$  — время обслуживания каждой заявки распределено экспоненциально с параметром  $\mu$ .
- $c$  — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

## 2.2.1 Основные параметры

- Интенсивность входящего потока ( $\lambda$ ): Среднее число заявок в единицу времени. Интервалы прибытия  $\sim \text{Exp}(1/\lambda)$
- Интенсивность обслуживания (одного канала) ( $\mu$ ): Среднее число заявок, обслуживаемых одним прибором в единицу времени. Длительность обслуживания  $\sim \text{Exp}(1/\mu)$
- Число каналов ( $c$ ): Количество параллельных серверов
- Загрузка системы на один канал (traffic intensity) ( $\rho = \lambda / (c \cdot \mu)$ ): Условие стационарности:  $\rho < 1$

По заданию требовалось полученный код перевести в структуру DrWatson - код был разделён на два - модель и базовый прогон.

Ниже предоставлен исходный код модели M/M/c, находящийся в папке /src:

```
using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
```

```

    t_a::Float64,
    d_s::Distribution,
    rng,
    log,
)

@yield timeout(env, t_a) # customer arrives
println("Customer $id arrived: ", now(env))
push!(log, (id=id, event="arrived", time=now(env)))

@yield request(server) # customer starts service
println("Customer $id entered service: ", now(env))
push!(log, (id=id, event="enteredService", time=now(env)))

@yield timeout(env, rand(rng, d_s)) # server is busy

@yield unlock(server) # customer exits service
println("Customer $id exited service: ", now(env))
push!(log, (id=id, event="exitedService", time=now(env)))
end

```

setup and run simulation

```

function setup_and_run(;
    seed=123,
    num_customers=10,
    num_servers=2,
    mu=1.0 / 2,
    lam=0.9,

```



```

)

rng = StableRNG(seed)

arrival_dist = Exponential(1 / lam) # interarrival time distribution
service_dist = Exponential(1 / mu) # service time distribution

sim = Simulation() # initialize simulation environment
server = Resource(sim, num_servers) # initialize servers

log = NamedTuple[]

arrival_time = 0.0

for i = 1:num_customers # initialize customers
    arrival_time += rand(rng, arrival_dist)
    @process customer(sim, server, i, arrival_time, service_dist)
end
run(sim) # run simulation
return log
end

```

Ниже представлен код базового прогона, создающий необходимые графики:

```

using DrWatson
@quickactivate "project"
include(sourcedir("mmc_model.jl"))

using DataFrames, CSV, Plots

```

```
# Запускаем симуляцию - задаём seed, количество клиентов, количество серверов
```

```
log = setup_and_run(seed=123, num_customers=10, num_servers=2, mu=1)
```

```
df = DataFrame(log)
```

Сохраняем данные в таблицу CSV

```
CSV.write(datadir("mmc_run.csv"), df)
```

Создаём график

```
times = Float64[]
```

```
clients_in_system = Int[]
```

```
current_clients = 0
```

```
for row in eachrow(df)
```

```
    if row.event == "arrived"
```

```
        global current_clients += 1
```

```
    elseif row.event == "exitedService"
```

```
        global current_clients -= 1
```

```
    end
```

```
    push!(times, row.time)
```

```
    push!(clients_in_system, current_clients)
```

```
end
```

```
system_df = DataFrame(
```

```
    time=times,
```

```
    clients_in_system=clients_in_system,
```

```

)

CSV.write(datadir("clients_in_system.csv"), system_df)

p = plot(
  system_df.time,
  system_df.clients_in_system,
  seriestype=:steppost,
  xlabel="Время (секунды)",
  ylabel="Число клиентов в системе",
  title="График изменения числа клиентов в системе M/M/c",
  legend=false,
)

savefig(p, plotsdir("mmc_events.png"))

println("График изменения числа клиентов в системе. Сохранен в data/mmc_run.csv")

```

Код работает успешно, были получены следующие график - число клиентов в системе на протяжении всего прогона модели (рис. 2.2).

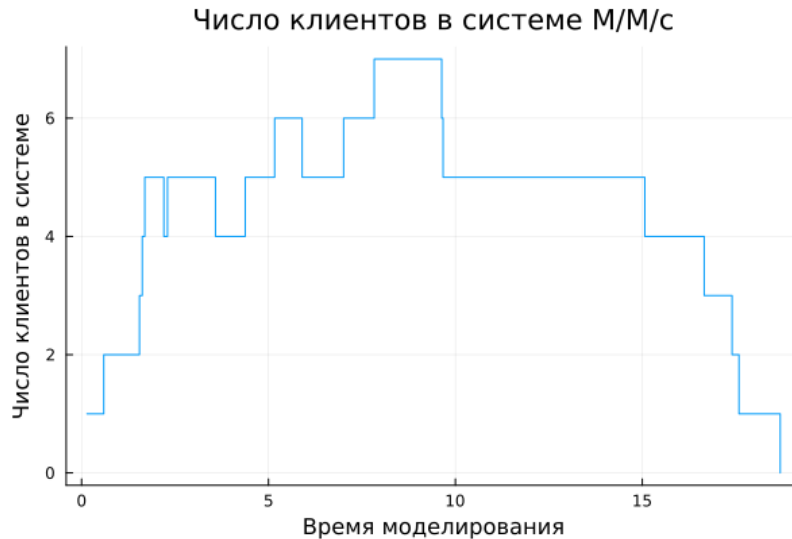


Рисунок 2.2: График динамики числа клиентов на протяжении всего прогона модели M/M/c

## 2.3 О модели Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

### 2.3.1 Постановка задачи

В системе находятся  $N$  идентичных машин, которые постоянно работают и могут выходить из строя.

$S$  машин находятся в резерве и готовы немедленно заменить любую отказавшую.

Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину.

Когда работающая машина ломается, происходит следующее:

- Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся.
- Сломанная машина отправляется в ремонт.

- Если резерва нет, система падает (crash). Моделирование заканчивается.

После ремонта машина пополняет пул резервных (становится исправной и ждёт).

Требуется оценить среднее время до падения системы при заданных распределениях наработки до отказа и времени ремонта.

### 2.3.2 Реализация в виде марковского процесса

Данная система может быть описана как марковский процесс с непрерывным временем и конечным числом состояний. Состояние можно определить как число работоспособных машин (работающих плюс резервных) или число машин в ремонте.

Пусть  $k$  — число исправных машин,  $0 \leq k \leq N + S$ . Классический подход из учебника Росса: состояние — число исправных машин, а число работающих всегда равно  $N$ , если есть хотя бы  $N$  исправных.

Состояния:

- $i$  — число исправных машин,  $i = 0, 1, \dots, N + S$ .
- Число работающих машин всегда  $\min(N, i)$ .
- Число резервных машин —  $\max(0, i - N)$ .
- Число машин в ремонте —  $N + S - i$ .

Интенсивности переходов:

- При отказе машины (если есть работающая) число исправных машин уменьшается на 1. Интенсивность:  $\min(N, i)\lambda$  (если  $i \geq 1$ ).
- При завершении ремонта число исправных увеличивается на 1. Интенсивность:  $\mu$ , если есть хотя бы одна машина в ремонте (т.е.  $i < N + S$ ). Ремонтник один, поэтому интенсивность постоянна, когда очередь на ремонт не пуста.

Система падает, когда число исправных машин  $i = N - 1$  (если ещё есть работающая) или  $i = N$ . Падение происходит в момент отказа, когда нет

резерва. Это соответствует переходу из состояния  $N$  (0 резерва, все  $N$  машин работают) в неработоспособное состояние. В момент отказа одной из  $N$  машин при  $i = N$  запасных нет, следовательно, система падает. Значит, падение наступает при первом достижении состояния  $i = N - 1$  (при переходе  $i = N \Rightarrow N - 1$ ).

### 2.3.3 Реализация модели на Julia

Для модели были заданы параметры по умолчанию следующим образом:

- $N = 10$  — основные машины;
- $S = 3$  — запасные машины;
- Нарботка до отказа одной машины  $\exp(\lambda)$ , где  $\lambda = 100$  (среднее время безотказной работы 100 часов);
- Время ремонта  $\exp(\mu)$ , где  $\mu = 1$  (среднее время ремонта 1 час).
- Машин много и они ломаются часто.
- Интенсивность отказов одной машины  $1/100 = 0.01$ , но так как их  $N = 10$ , суммарная интенсивность отказов работающих машин равна  $N * 0.01 = 0.1$  отказов в час.
- Ремонт относительно медленный (1 час на ремонт), поэтому резерв быстро истощается.
- В среднем каждые 10 часов ломается одна машина, за это время ремонтник успевает починить 10 машин, но начальный запас всего 3.
- При случайных флуктуациях быстро возникает ситуация, когда резерв исчерпывается.
- Увеличив  $S$  (число запасных), мы получаем значительно большее время падения.
- Например,  $S = 5$  может дать сотни часов.
- Это иллюстрирует важность резервирования.

По заданию требовалось полученный код перевести в структуру DrWatson - код был разделён на два - модель и базовый прогон.

Ниже предоставлен исходный код модели Росса, находящийся в папке /src:

```
using ResumableFunctions
using ConcurrentSim

using Distributions
using Random
using StableRNGs

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    rng,
    F::Distribution,
    G::Distribution,
    log,
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end

        @yield timeout(env, rand(rng, F))
        push!(log, (time=now(env), event="failure",))

        get_spare = take!(spares)
        @yield get_spare | timeout(env)
```

```

    if state(get_spare) != ConcurrentSim.idle
        @yield interrupt(value(get_spare))
        push!(log, (time=now(env), event="spare_used",))
    else
        push!(log, (time=now(env), event="crash",))
        throw(StopSimulation("No more spares!"))
    end
    @yield request(repair_facility)
    push!(log, (time=now(env), event="repair_start",))

    @yield timeout(env, rand(rng, G))

    @yield unlock(repair_facility)
    push!(log, (time=now(env), event="repair_end",))

    @yield put!(spares, active_process(env))
    push!(log, (time=now(env), event="spare_returned",))
end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    N::Int,
    S::Int,
    rng,
    F::Distribution,

```



```

        G::Distribution,
        log,
    )

    for i = 1:N
        proc = @process machine(env, repair_facility, spares, rng, 1)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares, rng, 2)
        @yield put!(spares, proc)
    end
end

function sim_repair(;
    N=10,
    S=3,
    num_repairmen=1,
    seed=42,
    lam=100.0,
    mu=1.0,
)
    rng = StableRNG(seed)

    F = Exponential(lam)
    G = Exponential(mu)

    sim = Simulation()

```

```

repair_facility = Resource(sim, num_repairmen)
spares = Store{Process}(sim)

log = NamedTuple[]

@process start_sim(sim, repair_facility, spares, N, S, rng, F, C)

msg = run(sim)
stop_time = now(sim)

return (
    stop_time=stop_time,
    msg=msg,
    log=log,
)
end

```

Ниже представлен код базового прогона модели, было увеличено количество ремонтников, был произведён прогон по разному количеству машин ( $N$ ), был проведён мониторинг загрузки ремонтника, средней длины очереди на ремонт, затем код создаёт графики изменения числа исправных машин во времени, а также несколько отрезков из этих графиков в увеличенном масштабе.

```

using DrWatson
@quickactivate ".."

include(sourcedir("ross_model.jl"))

using DataFrames

```

```

using CSV
using Plots
using Statistics

function build_repairmen_state(log_df, num_repairmen, stop_time)
    sort!(log_df, :time)

    times = [0.0]
    busy_repairmen = [0]
    current_busy = 0

    for row in eachrow(log_df)
        if row.event == "repair_start"
            current_busy += 1
            push!(times, row.time)
            push!(busy_repairmen, current_busy)
        elseif row.event == "repair_end"
            current_busy -= 1
            push!(times, row.time)
            push!(busy_repairmen, current_busy)
        end
    end

    push!(times, stop_time)
    push!(busy_repairmen, current_busy)

    state_df = DataFrame(time=times, busy_repairmen=busy_repairmen)

```

```

busy_area = 0.0

for i in 1:(nrow(state_df) - 1)
    dt = state_df.time[i + 1] - state_df.time[i]
    busy_area += state_df.busy_repairmen[i] * dt
end

utilization = busy_area / (stop_time * num_repairmen)

return state_df, utilization
end

function build_repair_queue_state(log_df, stop_time)
    sort!(log_df, :time)

    times = [0.0]
    queue_lengths = [0]
    current_queue = 0

    for row in eachrow(log_df)
        if row.event == "repair_request"
            current_queue += 1
            push!(times, row.time)
            push!(queue_lengths, current_queue)
        elseif row.event == "repair_start"
            current_queue -= 1
            push!(times, row.time)
            push!(queue_lengths, current_queue)
        end
    end
end

```

```

        end

    end

    push!(times, stop_time)
    push!(queue_lengths, current_queue)

    queue_df = DataFrame(time=times, queue_length=queue_lengths)

    queue_area = 0.0

    for i in 1:(nrow(queue_df) - 1)
        dt = queue_df.time[i + 1] - queue_df.time[i]
        queue_area += queue_df.queue_length[i] * dt
    end

    mean_queue_length = queue_area / stop_time

    return queue_df, mean_queue_length
end

function build_good_machines_state(log_df, N, S, stop_time)
    sort!(log_df, :time)

    times = [0.0]
    good_machines = [N + S]
    current_good = N + S

    for row in eachrow(log_df)

```

```

        if row.event == "failure"
            current_good -= 1
            push!(times, row.time)
            push!(good_machines, current_good)
        elseif row.event == "repair_end"
            current_good += 1
            push!(times, row.time)
            push!(good_machines, current_good)
        end
    end

    push!(times, stop_time)
    push!(good_machines, current_good)

    return DataFrame(time=times, good_machines=good_machines)
end

RUNS = 5

Ns = [5, 10, 15, 20]
S = 3
NUM_REPAIRMEN = 3

results = DataFrame(
    N=Int[],
    S=Int[],
    num_repairmen=Int[],
    run=Int[],

```

```

        crash_time=Float64[],
        utilization=Float64[],
        mean_queue_length=Float64[],
    )

all_good_machines = DataFrame(
    N=Int[],
    S=Int[],
    num_repairmen=Int[],
    run=Int[],
    time=Float64[],
    good_machines=Int[],
)

for N in Ns
    for run_id in 1:RUNS
        result = sim_repair(
            N=N,
            S=S,
            num_repairmen=NUM_REPAIRMEN,
            seed=100 + run_id,
            lam=100.0,
            mu=1.0,
        )

        log_df = DataFrame(result.log)

        repairmen_df, utilization = build_repairmen_state(

```

```

        log_df,
        NUM_REPAIRMEN,
        result.stop_time,
    )

    queue_df, mean_queue_length = build_repair_queue_state(
        log_df,
        result.stop_time,
    )

    good_df = build_good_machines_state(
        log_df,
        N,
        S,
        result.stop_time,
    )

    good_df.N .= N
    good_df.S .= S
    good_df.num_repairmen .= NUM_REPAIRMEN
    good_df.run .= run_id

    append!(
        all_good_machines,
        good_df[:, [:N, :S, :num_repairmen, :run, :time, :good_m]],
    )

    push!(results, (

```



```

        N=N,
        S=S,
        num_repairmen=NUM_REPAIRMEN,
        run=run_id,
        crash_time=result.stop_time,
        utilization=utilization,
        mean_queue_length=mean_queue_length,
    ))

    if N == 10 && run_id == 1
        CSV.write(datadir("ross", "log_example.csv"), log_df)
        CSV.write(datadir("ross", "repairmen_state_example.csv"),
        CSV.write(datadir("ross", "repair_queue_example.csv"),
    end
end
end

CSV.write(datadir("ross_results.csv"), results)
CSV.write(datadir("all_good_machines.csv"), all_good_machines)

summary = combine(
    groupby(results, [:N, :num_repairmen]),
    :crash_time => mean => :mean_crash_time,
    :crash_time => std => :std_crash_time,
    :utilization => mean => :mean_utilization,
    :mean_queue_length => mean => :mean_queue_length,
)

```

```

CSV.write(datadir("ross", "ross_summary.csv"), summary)

# 00000000

one_run = filter(
  row -> row.N == 10 && row.run == 1,
  all_good_machines,
)

sort!(one_run, :time)

CSV.write(datadir("ross", "good_machines_one_run.csv"), one_run)

p1 = plot(
  one_run.time,
  one_run.good_machines,
  seriestype=:steppost,
  xlabel="000000 0000000000000000",
  ylabel="000000 0000000000 000000",
  title="0000000000 0000000000 0000000000 000000",
  legend=false,
  xlims=(0, 80),
  ylims=(10, 14),
)

savefig(p1, plotsdir("machines_one_run_small_1.png"))

p2 = plot(

```

```

    one_run.time,
    one_run.good_machines,
    seriestype=:steppost,
    xlabel="Time (seconds)",
    ylabel="Number of good machines",
    title="Number of good machines over time",
    legend=false,
    xlims=(0, 1000),
    ylims=(10, 14),
)

savefig(p2, plotsdir("machines_one_run_large.png"))

p3 = plot(
    one_run.time,
    one_run.good_machines,
    seriestype=:steppost,
    xlabel="Time (seconds)",
    ylabel="Number of good machines",
    title="Number of good machines over time",
    legend=false,
    xlims=(200, 300),
    ylims=(10, 14),
)

savefig(p3, plotsdir("machines_one_run_small_2.png"))

```

```
println(summary)
```

В результате были получены следующие графики:

График динамика числа исправных машин (рис. 2.3).

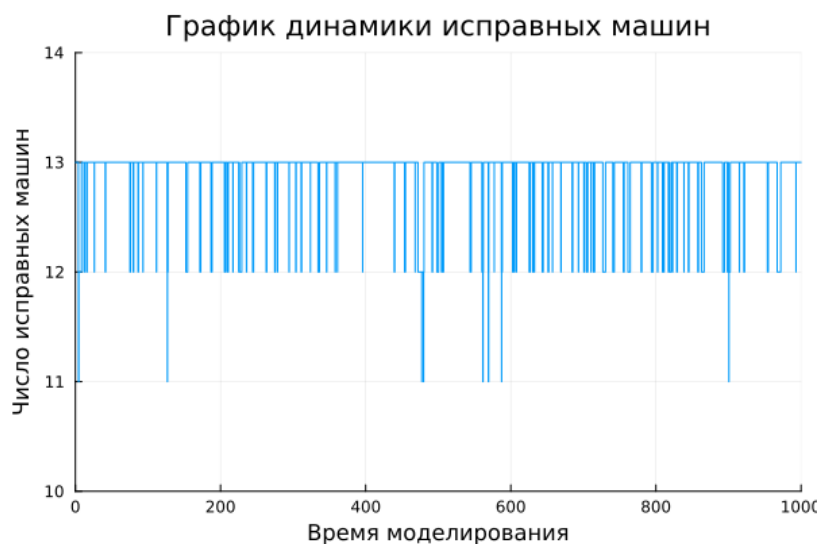


Рисунок 2.3: График динамики числа исправных машин на протяжении всего прогона модели Росса

Следующие два графика аналогичны предыдущему, но был увеличен масштаб, чтобы детальнее рассмотреть динамику (рис. 2.4 - Начало), (рис. 2.5 - середина).

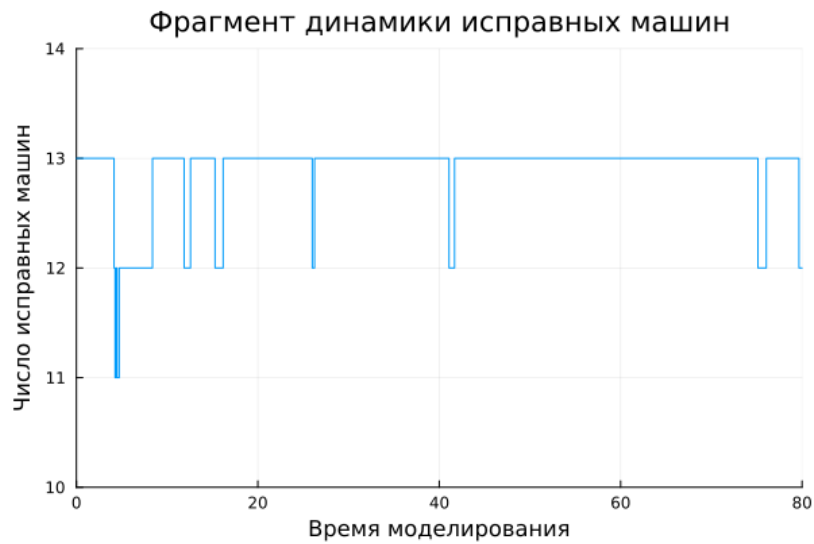


Рисунок 2.4: Начало графика динамики числа исправных машин на протяжении всего прогона модели Росса

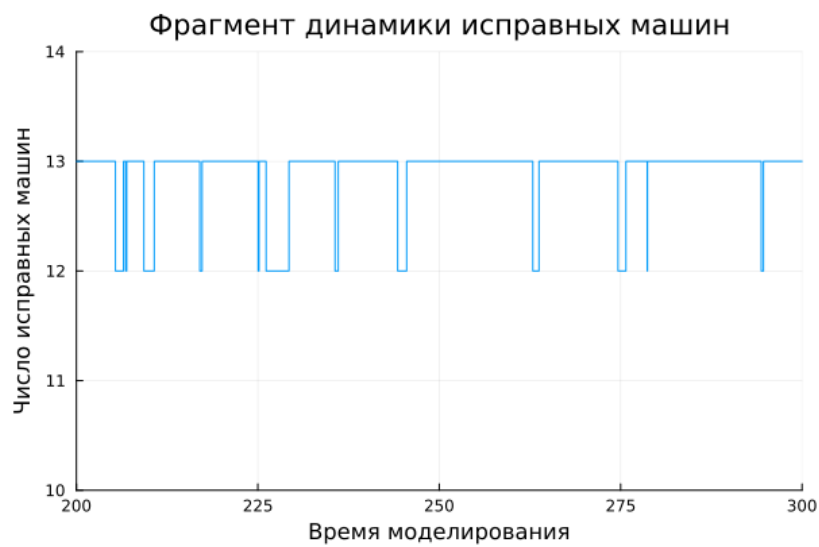


Рисунок 2.5: Середина графика динамики числа исправных машин на протяжении всего прогона модели Росса

Также после работы кода была получена следующая статистика работы модели (рис. 2.6).

```
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab07\project> julia --project
```

4x6 DataFrame

| Row | N     | num_repairmen | mean_crash_time | std_crash_time | mean_utilization | mean_queue_length |
|-----|-------|---------------|-----------------|----------------|------------------|-------------------|
|     | Int64 | Int64         | Float64         | Float64        | Float64          | Float64           |
| 1   | 5     | 3             | 4.32045e5       | 3.08153e5      | 0.0169309        | -10830.8          |
| 2   | 10    | 3             | 30395.7         | 13092.9        | 0.0330121        | -1515.45          |
| 3   | 15    | 3             | 8396.38         | 5643.2         | 0.0497436        | -639.408          |
| 4   | 20    | 3             | 5080.73         | 3808.41        | 0.0682295        | -509.966          |

```
PS C:\Users\12232\Documents\GitHub\2026-1--study--simulation-modeling\labs\lab07\project>
```

Рисунок 2.6: Статистика прогона модели Росса

## 3 Выводы

В результате выполнения данной лабораторной работы я укрепил свои знания в работе DrWatson, адаптировав полученный код к необходимому формату, а также получил навыки в работе с Julia, написав код для создания графиков и выполнения дополнительных заданий.

## 4 Список литературы

Лабораторная работа №7